

Algorithm Part-01

Floyd Warshall Algorithm:

The **Floyd Warshall Algorithm** is an all pair shortest path algorithm unlike Dijkstra and Bellman Ford which are single source shortest path algorithms. This algorithm works for both the **directed** and **undirected weighted** graphs. But, it does not work for the graphs with negative cycles (where the sum of the edges in a cycle is negative).

Idea Behind Floyd Warshall Algorithm:

Suppose we have a graph $G[V][V]$ with V vertices from 1 to N . Now we have to evaluate a **shortestPathMatrix** where **shortestPathMatrix[i][j]** represents the shortest path between vertices i and j .

Obviously the shortest path between i to j will have some k number of intermediate nodes. The idea behind floyd warshall algorithm is to treat each and every vertex from 1 to N as an intermediate node one by one.

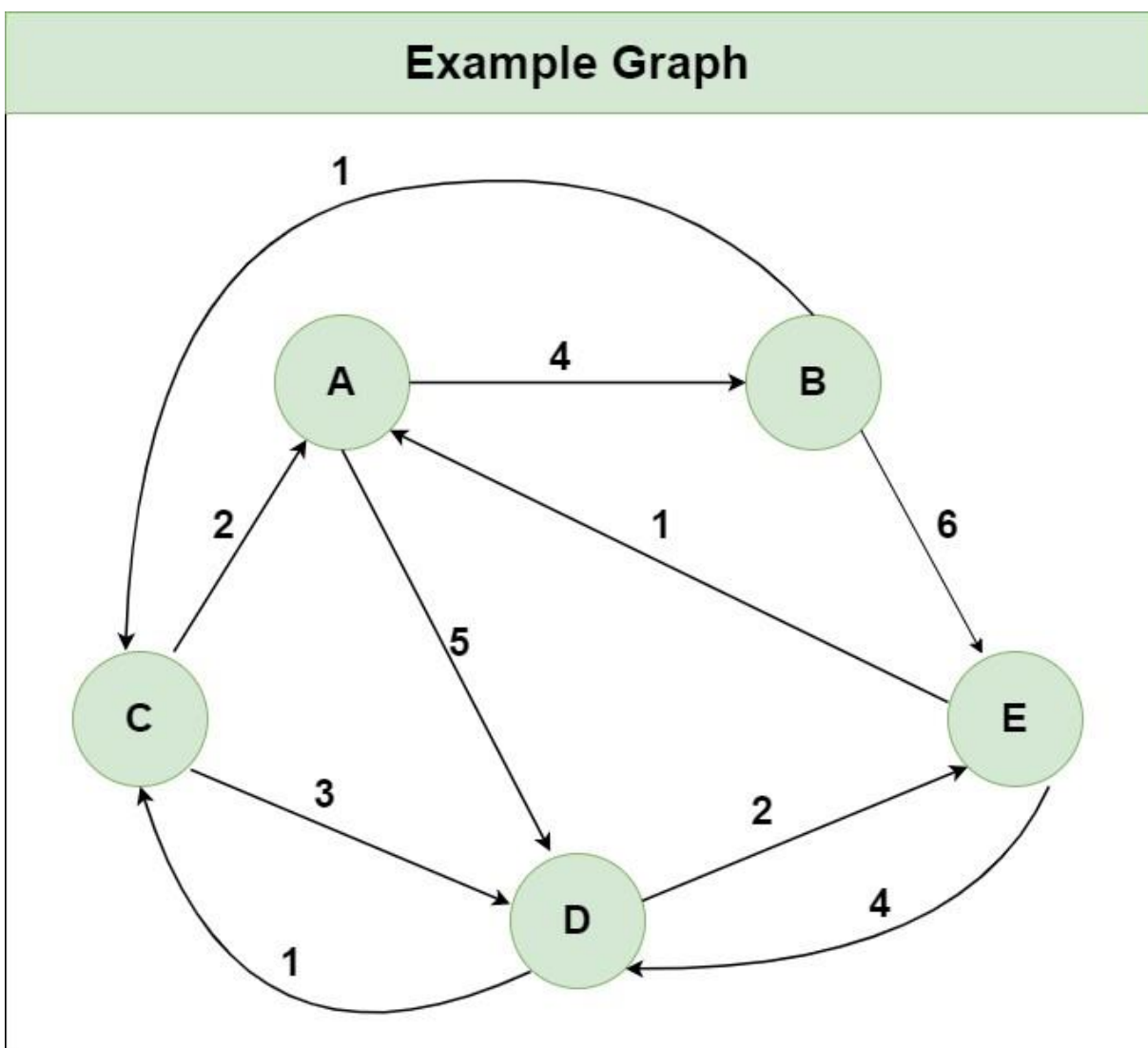
Floyd Warshall Algorithm:

- Initialize the solution matrix same as the input graph matrix as a first step.
- Then update the solution matrix by considering all vertices as an intermediate vertex.
- The idea is to pick all vertices one by one and updates all shortest paths which include the picked vertex as an intermediate vertex in the shortest path.
- When we pick vertex number k as an intermediate vertex, we already have considered vertices $\{0, 1, 2, \dots, k-1\}$ as intermediate vertices.
- For every pair (i, j) of the source and destination vertices respectively, there are two possible cases.

- **k** is not an intermediate vertex in shortest path from **i** to **j**. We keep the value of **dist[i][j]** as it is.
- **k** is an intermediate vertex in shortest path from **i** to **j**. We update the value of **dist[i][j]** as **dist[i][k] + dist[k][j]**, if **dist[i][j] > dist[i][k] + dist[k][j]**

Illustration of Floyd Warshall Algorithm :

Suppose we have a graph as shown in the image:



Step 1: Initialize the Distance[][] matrix using the input graph such that Distance[i][j] = weight of edge from i to j, also Distance[i][j] = Infinity if there is no edge from i to j.

Step1: Initializing Distance[][] using the Input Graph					
	A	B	C	D	E
A	0	4	∞	5	∞
B	∞	0	1	∞	6
C	2	∞	0	3	∞
D	∞	∞	1	0	2
E	1	∞	∞	4	0

Step 2: Treat node A as an intermediate node and calculate the Distance[][] for every {i,j} node pair using the formula:

= Distance[i][j] = minimum (Distance[i][j], (Distance from i to A) + (Distance from A to j))

= Distance[i][j] = minimum (Distance[i][j], Distance[i][A] + Distance[A][j])

Step 2: Using Node A as the Intermediate node					
Distance[i][j] = min (Distance[i][j], Distance[i][A] + Distance[A][j])					
	A	B	C	D	E
A	0	4	∞	5	∞
B	∞	?	?	?	?
C	2	?	?	?	?
D	∞	?	?	?	?
E	1	?	?	?	?

→

	A	B	C	D	E
A	0	4	∞	5	∞
B	∞	0	1	∞	6
C	2	6	0	3	INF
D	∞	∞	1	0	2
E	1	5	∞	4	0

Step 3: Treat node **B** as an intermediate node and calculate the Distance[][] for every {i,j} node pair using the formula:

= Distance[i][j] = minimum (Distance[i][j], (Distance from i to **B**) + (Distance from **B** to j))

= Distance[i][j] = minimum (Distance[i][j], Distance[i][**B**] + Distance[**B**][j])

Step 3: Using Node B as the Intermediate node					
Distance[i][j] = min (Distance[i][j], Distance[i][B] + Distance[B][j])					
	A	B	C	D	E
A	?	4	?	?	?
B	∞	0	1	∞	6
C	?	6	?	?	?
D	?	∞	?	?	?
E	?	5	?	?	?

	A	B	C	D	E
A	0	4	5	5	10
B	∞	0	1	∞	6
C	2	6	0	3	12
D	∞	∞	1	0	2
E	1	5	6	4	0

Step 4: Treat node **C** as an intermediate node and calculate the Distance[][] for every {i,j} node pair using the formula:

= Distance[i][j] = minimum (Distance[i][j], (Distance from i to **C**) + (Distance from **C** to j))

= Distance[i][j] = minimum (Distance[i][j], Distance[i][**C**] + Distance[**C**][j])

Step 4: Using Node C as the Intermediate node					
Distance[i][j] = min (Distance[i][j], Distance[i][C] + Distance[C][j])					
	A	B	C	D	E
A	?	?	5	?	?
B	?	?	1	?	?
C	2	6	0	3	12
D	?	?	1	?	?
E	?	?	6	?	?

	A	B	C	D	E
A	0	4	5	5	10
B	3	0	1	4	6
C	2	6	0	3	INF
D	3	7	1	0	2
E	1	5	6	4	0

Step 5: Treat node **D** as an intermediate node and calculate the Distance[i][j] for every {i,j} node pair using the formula:

= Distance[i][j] = minimum (Distance[i][j], (Distance from i to **D**) + (Distance from **D** to j))

= Distance[i][j] = minimum (Distance[i][j], Distance[i][**D**] + Distance[**D**][j])

Step 5: Using Node D as the Intermediate node					
Distance[i][j] = min (Distance[i][j], Distance[i][D] + Distance[D][j])					
	A	B	C	D	E
A	?	?	?	5	?
B	?	?	?	4	?
C	?	?	?	3	?
D	3	7	1	0	2
E	?	?	?	4	?

	A	B	C	D	E
A	0	4	5	5	7
B	3	0	1	4	6
C	2	6	0	3	5
D	3	7	1	0	2
E	1	5	5	4	0

Step 6: Treat node **E** as an intermediate node and calculate the Distance[i][j] for every {i,j} node pair using the formula:

= Distance[i][j] = minimum (Distance[i][j], (Distance from i to **E**) + (Distance from **E** to j))

$$= \text{Distance}[i][j] = \text{minimum} (\text{Distance}[i][j], \text{Distance}[i][E] + \text{Distance}[E][j])$$

Step 6: Using Node E as the Intermediate node						
Distance[i][j] = min (Distance[i][j], Distance[i][E] + Distance[E][j])						
	A	B	C	D	E	
A	?	?	?	?	7	
B	?	?	?	?	6	
C	?	?	?	?	5	
D	?	?	?	?	2	
E	1	5	5	4	0	

	A	B	C	D	E	
A	0	4	5	5	7	
B	3	0	1	4	6	
C	2	6	0	3	5	
D	3	7	1	0	2	
E	1	5	5	4	0	

Step 7: Since all the nodes have been treated as an intermediate node, we can now return the updated Distance[][] matrix as our answer matrix.

Step 7: Return Distance[][] matrix as the result						
	A	B	C	D	E	
A	0	4	5	5	7	
B	3	0	1	4	6	
C	2	6	0	3	5	
D	3	7	1	0	2	
E	1	5	5	4	0	

Pseudo-Code of Floyd Warshall Algorithm :

For $k = 0$ to $n - 1$

For $i = 0$ to $n - 1$

For $j = 0$ to $n - 1$

$\text{Distance}[i, j] = \min(\text{Distance}[i, j], \text{Distance}[i, k] + \text{Distance}[k, j])$ where

i = source Node, j = Destination Node, k = Intermediate Node

Complexity Analysis of Floyd-Warshall Algorithm:

Time complexity: $O(n^3)$, where n is the number of vertices

Auxiliary Space: $O(n^2)$

The Floyd-Warshall algorithm has a time complexity of $O(n^3)$ for all cases, as it has three nested loops, each iterating through all the vertices.

The algorithm uses an $O(n^2)$ adjacency matrix to store the shortest distances between all pairs of vertices.

Bellman-Ford Algorithm

Bellman-Ford is a **single source** shortest path algorithm that determines the shortest path between a given source vertex and every other vertex in a graph. This algorithm can be used on both **weighted** and **unweighted** graphs.

A **Bellman-Ford** algorithm is also certain to find the shortest path in a graph, similar to Dijkstra's Algorithm. Although Bellman-Ford is slower than **Dijkstra's algorithm**, it is capable of handling graphs with **negative edge weights**, which makes it more versatile. The shortest path cannot be found if there exists a **negative cycle** in the graph. If we continue to go around the negative cycle an infinite number of times, then the cost of the path will continue to decrease (even though the length of the path is increasing). As a result, **Bellman-Ford** is also capable of detecting **negative cycles**, which is an important feature.

The idea behind Bellman Ford Algorithm:

The Bellman-Ford algorithm's primary principle is that it starts with a single source and calculates the distance to each node. The distance is initially unknown and assumed to be infinite, but as time goes on, the algorithm relaxes those paths by identifying a few shorter paths. Hence it is said that BellmanFord is based on "**Principle of Relaxation**".

Principle of Relaxation of Edges for Bellman-Ford:

- It states that for the graph having **N** vertices, all the edges should be relaxed **N-1** times to compute the single source shortest path.
- In order to detect whether a negative cycle exists or not, relax all the edge one more time and if the shortest distance for any node reduces then we can say that a negative cycle exists. In short if we relax the edges **N** times, and there is any change in the shortest distance of any node between the **N-1th** and **Nth** relaxation than a negative cycle exists, otherwise not exist.

Why Relaxing Edges N-1 times, gives us Single Source Shortest Path?

The Bellman-Ford algorithm can find the shortest paths from a starting vertex to all other vertices in a graph.

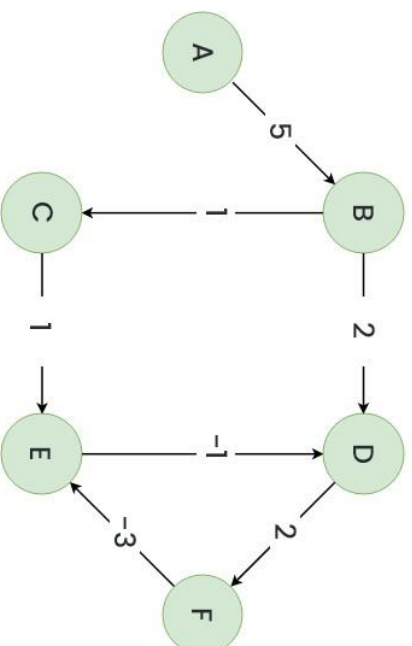
The algorithm does this by relaxing, or updating, the distances to each vertex up to $N-1$ times, where N is the number of vertices in the graph. This is because the longest possible shortest path between any two vertices in a graph with N vertices has $N-1$ edges. If the graph has a negative-weight cycle that can be reached from the starting vertex, the algorithm can detect this after $N-1$ iterations, as the negative cycle will disrupt the shortest path lengths. So, by relaxing the edges $N-1$ times, the Bellman-Ford algorithm ensures it has found the true shortest paths, as long as there are no negative-weight cycles.

Why Does the Reduction of Distance in the N'th Relaxation Indicates the Existence of a Negative Cycle?

As previously discussed, achieving the single source shortest paths to all other nodes takes $N-1$ relaxations. If the N 'th relaxation further reduces the shortest distance for any node, it implies that a certain edge with negative weight has been traversed once more. It is important to note that during the $N-1$ relaxations, we presumed that each vertex is traversed only once. However, the reduction of distance during the N 'th relaxation indicates revisiting a vertex.

Working of Bellman-Ford Algorithm to Detect the Negative cycle in the graph:

Let's suppose we have a graph which is given below and we want to find whether there exists a negative cycle or not using Bellman-Ford.



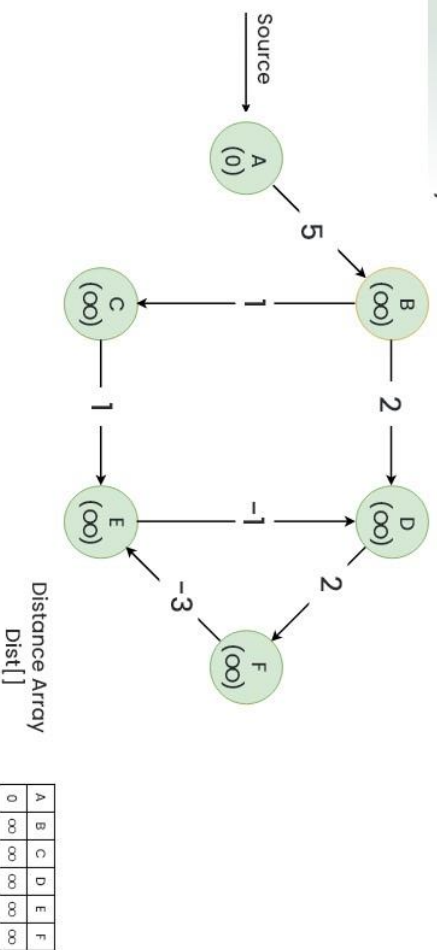
Bellman-Ford To Detect A Negative Cycle In A Graph



Initial Graph

Step 1: Initialize a distance array `Dist[]` to store the shortest distance for each vertex from the source vertex. Initially distance of source will be 0 and Distance of other vertices will be INFINITY.

Initialize The Distance Array



Bellman-Ford To Detect A Negative Cycle In A Graph



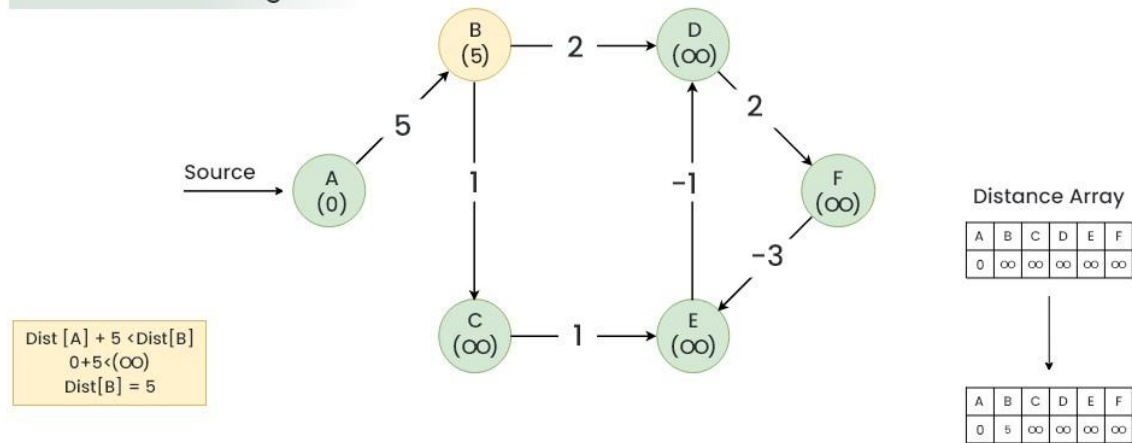
Initialize a distance array

Step 2: Start relaxing the edges, during 1st Relaxation:

- Current Distance of B > (Distance of A) + (Weight of A to B) i.e. Infinity > 0 + 5

- Therefore, $\text{Dist}[B] = 5$

1st Relaxation Of Edges



Bellman-Ford To Detect A Negative Cycle In A Graph

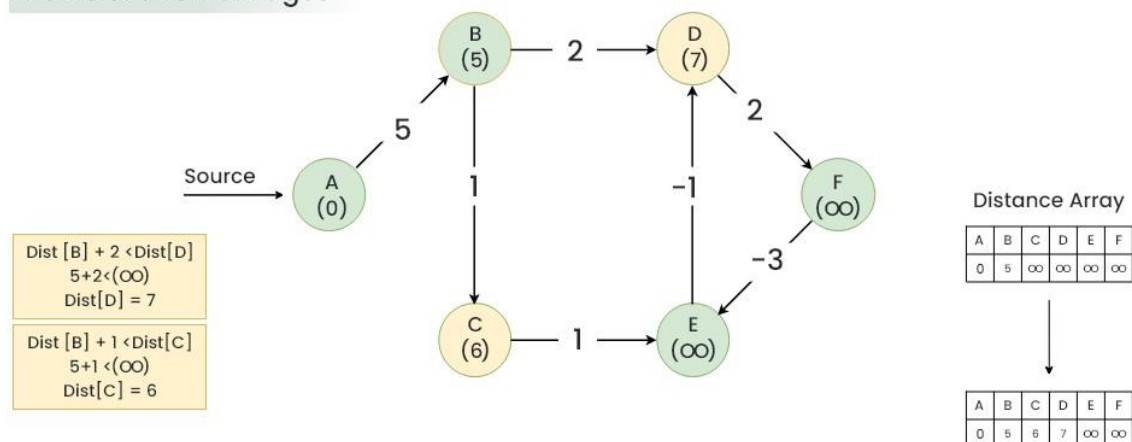


1st Relaxation

Step 3: During 2nd Relaxation:

- Current Distance of D > (Distance of B) + (Weight of B to D) i.e.
 $\text{Infinity} > 5 + 2$ ∴ $\text{Dist}[D] = 7$
- Current Distance of C > (Distance of B) + (Weight of B to C) i.e.
 $\text{Infinity} > 5 + 1$ ∴ $\text{Dist}[C] = 6$

2nd Relaxation Of Edges



Bellman-Ford To Detect A Negative Cycle In A Graph

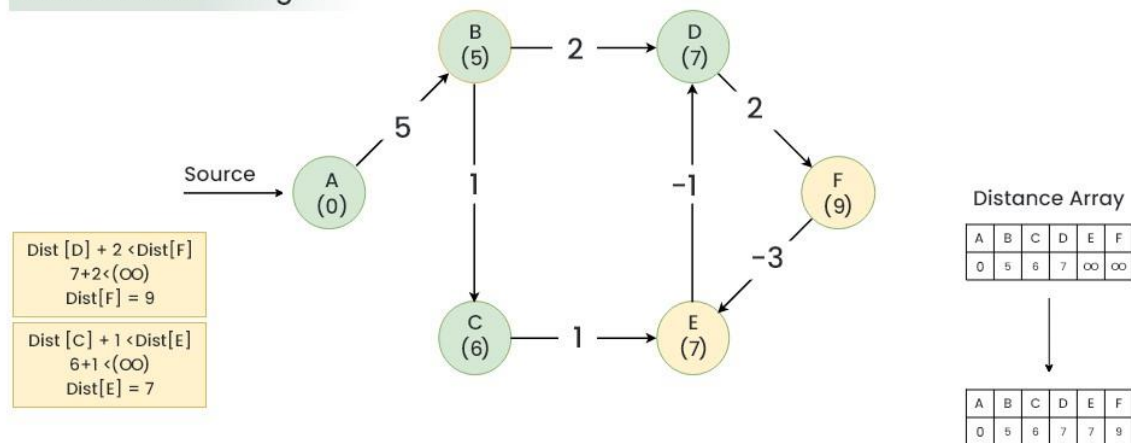


2nd Relaxation

Step 4: During 3rd Relaxation:

- Current Distance of F > (Distance of D) + (Weight of D to F) i.e.
Infinity > 7 + 2
 - Dist[F] = 9
- Current Distance of E > (Distance of C) + (Weight of C to E) i.e.
Infinity > 6 + 1 ◦ Dist[E] = 7

3rd Relaxation Of Edges



Bellman-Ford To Detect A Negative Cycle In A Graph

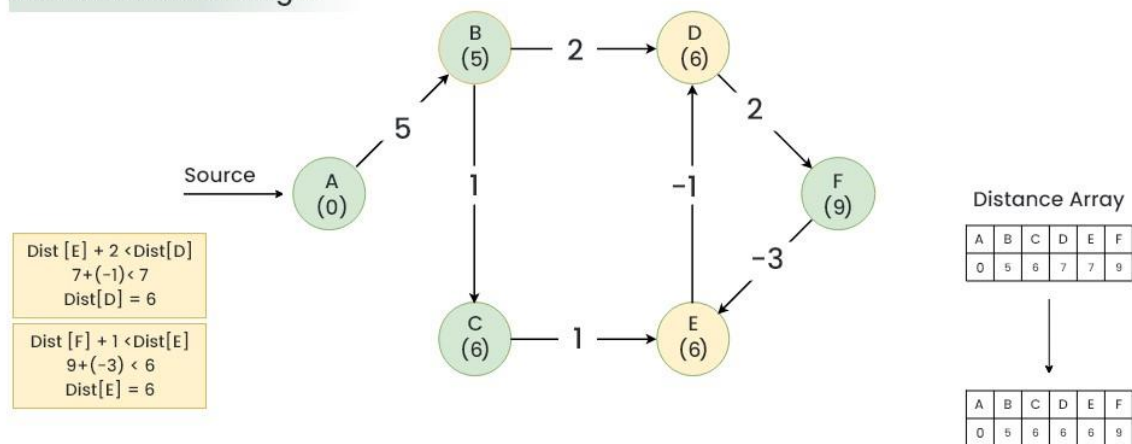


3rd Relaxation

Step 5: During 4th Relaxation:

- Current Distance of D > (Distance of E) + (Weight of E to D) i.e. 7 > 7 + (-1) ◦ Dist[D] = 6
- Current Distance of E > (Distance of F) + (Weight of F to E) i.e. 7 > 9 + (-3) ◦ Dist[E] = 6

4th Relaxation Of Edges



Bellman-Ford To Detect A Negative Cycle In A Graph

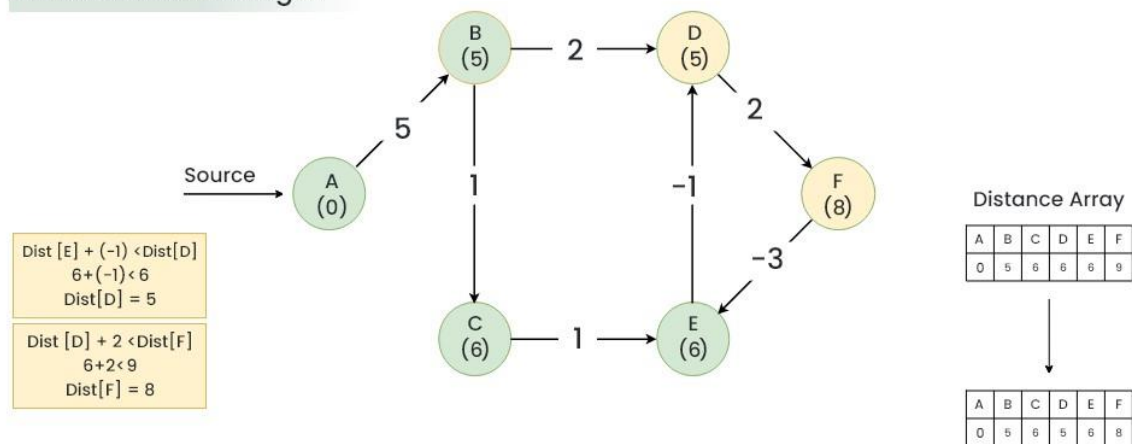


4th Relaxation

Step 6: During 5th Relaxation:

- Current Distance of F > (Distance of D) + (Weight of D to F) i.e. $9 > 6 + 2$ ∴ Dist[F] = 8
- Current Distance of D > (Distance of E) + (Weight of E to D) i.e. $6 > 6 + (-1)$ ∴ Dist[D] = 5
- Since the graph has 6 vertices, So during the 5th relaxation the shortest distance for all the vertices should have been calculated.

5th Relaxation Of Edges



Bellman-Ford To Detect A Negative Cycle In A Graph



5th Relaxation

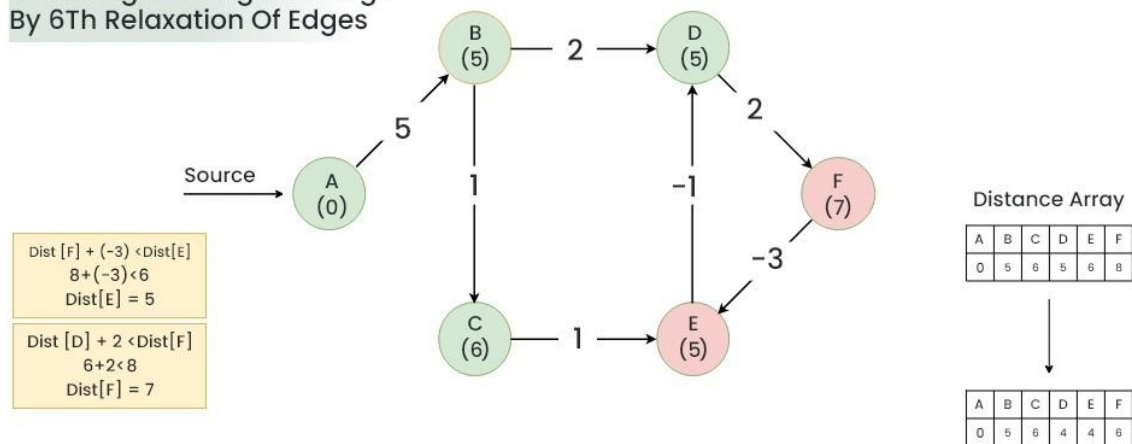
Step 7: Now the final relaxation i.e. the 6th relaxation should indicate the presence of negative cycle if there is any changes in the distance array of 5th relaxation.

During the 6th relaxation, following changes can be seen:

- Current Distance of E > (Distance of F) + (Weight of F to E) i.e. $6 > 8 + (-3)$
 - Dist[E]=5
- Current Distance of F > (Distance of D) + (Weight of D to F) i.e. $8 > 5 + 2$
 - Dist[F]=7

Since, we observe changes in the Distance array Hence, we can conclude the presence of a negative cycle in the graph.

Detecting The Negative Edge By 6Th Relaxation Of Edges



Bellman-Ford To Detect A Negative Cycle In A Graph



6th Relaxation

Result: A negative cycle (D->F->E) exists in the graph.

Algorithm to Find Negative Cycle in a Directed Weighted Graph Using Bellman-Ford:

- Initialize distance array $\text{dist}[]$ for each vertex 'v' as $\text{dist}[v] = \text{INFINITY}$.
- Assume any vertex (let's say '0') as source and assign $\text{dist} = 0$.
- Relax all the **edges(u,v,weight)** N-1 times as per the below condition:
 - $\text{dist}[v] = \text{minimum}(\text{dist}[v], \text{distance}[u] + \text{weight})$
- Now, Relax all the edges one more time i.e. the **Nth** time and based on the below two cases we can detect the negative cycle:
 - Case 1 (Negative cycle exists): For any **edge(u, v, weight)**, if $\text{dist}[u] + \text{weight} < \text{dist}[v]$
 - Case 2 (No Negative cycle) : case 1 fails for all the edges.

Handling Disconnected Graphs in the Algorithm:

The above algorithm and program might not work if the given graph is disconnected. It works when all vertices are reachable from source vertex **0**. To handle disconnected graphs, we can repeat the above algorithm for vertices having **distance = INFINITY**, or simply for the vertices that are not visited.

Pseudocode:

ALGORITHM: Bellman-Ford

INPUT: n, m, u, v, w

INITIALIZE:

 dist (set all to infinity)

 dist[start] = 0

REPEAT (n-1) TIMES:

 for each edge (u, v, w) in the graph:

 if dist[v] > dist[u] + w:

 dist[v] = dist[u] + w

CHECK NEGATIVE CYCLE:

 for each edge (u, v, w) in the graph:

 if dist[v] > dist[u] + w:

 print "Negative Cycle Detected"

 return

PRINT dist

Complexity Analysis of Bellman-Ford Algorithm:

- **Time Complexity when graph is connected:**
 - **Best Case:** $O(E)$, when distance array after 1st and 2nd relaxation are same , we can simply stop further processing
 - **Average Case:** $O(V * E)$
 - **Worst Case:** $O(V * E)$
- **Time Complexity when graph is disconnected:**
 - **All the cases:** $O(E * (V^2))$
- **Auxiliary Space:** $O(V)$, where V is the number of vertices in the graph.

Bellman Ford's Algorithm Applications:

- **Network Routing:** Bellman-Ford is used in computer networking to find the shortest paths in routing tables, helping data packets navigate efficiently across networks.
- **GPS Navigation:** GPS devices use Bellman-Ford to calculate the shortest or fastest routes between locations, aiding navigation apps and devices.
- **Transportation and Logistics:** Bellman-Ford's algorithm can be applied to determine the optimal paths for vehicles in transportation and logistics, minimizing fuel consumption and travel time.
- **Game Development:** Bellman-Ford can be used to model movement and navigation within virtual worlds in game development, where different paths may have varying costs or obstacles.
- **Robotics and Autonomous Vehicles:** The algorithm aids in path planning for robots or autonomous vehicles, considering obstacles, terrain, and energy consumption.

Drawback of Bellman Ford's Algorithm:

- Bellman-Ford algorithm will fail if the graph contains any negative edge cycle.

Dijkstra's Algorithm

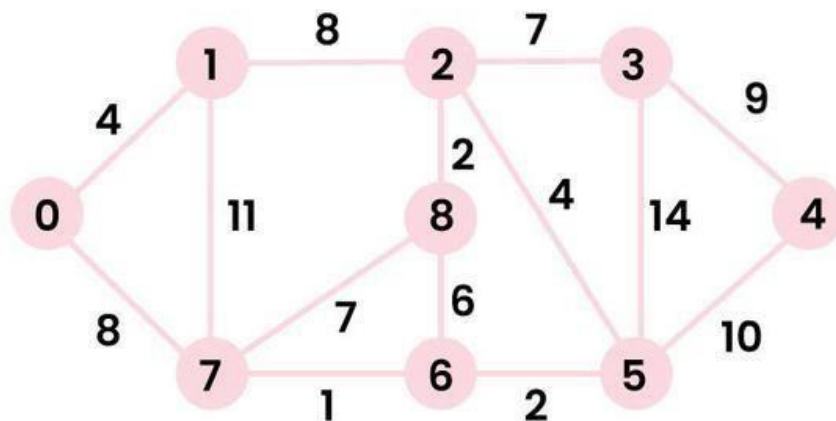
How to find Shortest Paths from Source to all Vertices using Dijkstra's Algorithm

Given a weighted graph and a source vertex in the graph, find the **shortest paths** from the source to all the other vertices in the given graph.

Note: The given graph does not contain any negative edge.

Examples:

Input: src = 0, the graph is shown below.



Working of Dijkstra's Algorithm



Output: 0 4 12 19 21 11 9 8 14

Explanation: The distance from 0 to 1 = 4.

The minimum distance from 0 to 2 = 12. 0->1->2

The minimum distance from 0 to 3 = 19. 0->1->2->3

The minimum distance from 0 to 4 = 21. 0->7->6->5->4

The minimum distance from 0 to 5 = 11. 0->7->6->5

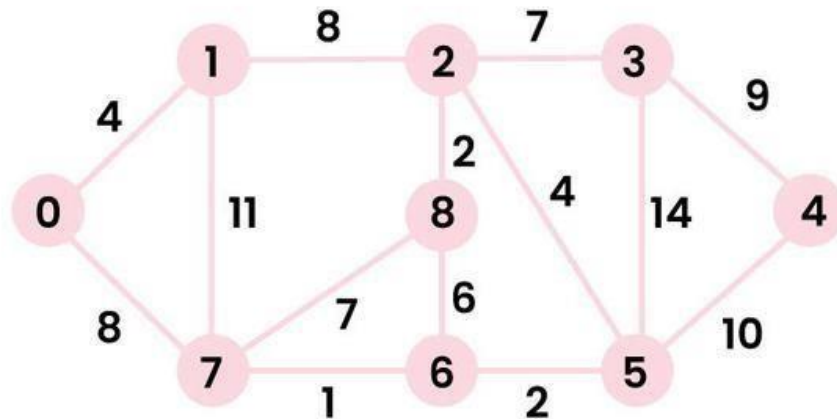
The minimum distance from 0 to 6 = 9. 0->7->6

The minimum distance from 0 to 7 = 8. 0->7

The minimum distance from 0 to 8 = 14. 0->1->2->8

Illustration of Dijkstra Algorithm:

To understand the Dijkstra's Algorithm let's take a graph and find the shortest path from source to all nodes. Consider below graph and **src = 0**
SPT(Shortest path tree).



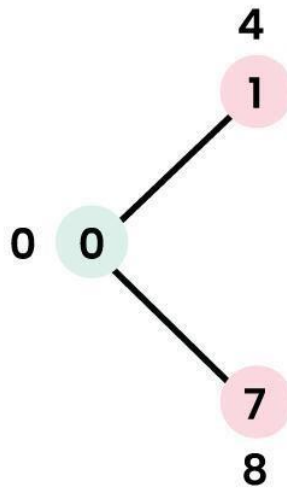
Working of Dijkstra's Algorithm



Step 1:

- The set **sptSet** is initially empty and distances assigned to vertices are {0, INF, INF, INF, INF, INF, INF, INF, INF} where **INF** indicates infinite.
- Now pick the vertex with a minimum distance value. The vertex 0 is picked, include it in **sptSet**. So **sptSet** becomes {0}. After including 0 to **sptSet**, update distance values of its adjacent vertices.
- Adjacent vertices of 0 are 1 and 7. The distance values of 1 and 7 are updated as 4 and 8.

The following subgraph shows vertices and their distance values.

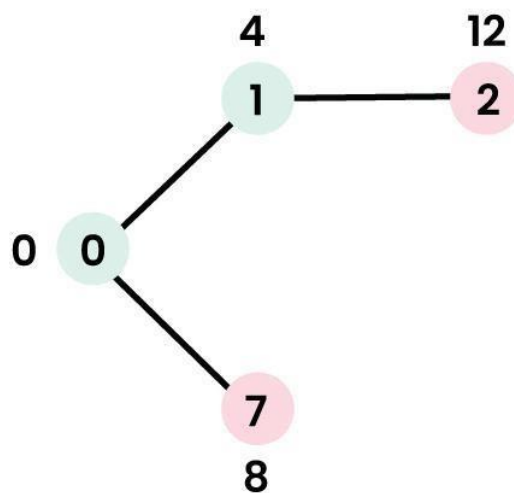


Working of Dijkstra's Algorithm



Step 2:

- Pick the vertex with minimum distance value and not already included in **SPT** (not in **sptSet**). The vertex 1 is picked and added to **sptSet**.
- So **sptSet** now becomes {0, 1}. Update the distance values of adjacent vertices of 1.
- The distance value of vertex 2 becomes **12**.

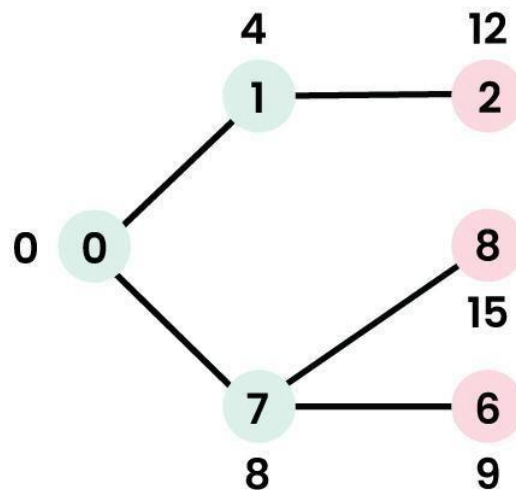


Working of Dijkstra's Algorithm



Step 3:

- Pick the vertex with minimum distance value and not already included in **SPT** (not in **sptSET**). Vertex 7 is picked. So **sptSet** now becomes **{0, 1, 7}**.
- Update the distance values of adjacent vertices of 7. The distance value of vertex 6 and 8 becomes finite (**15 and 9** respectively).

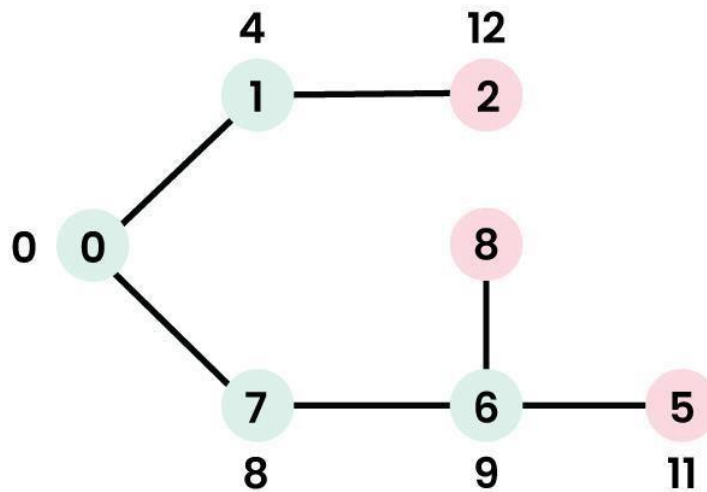


Working of Dijkstra's Algorithm



Step 4:

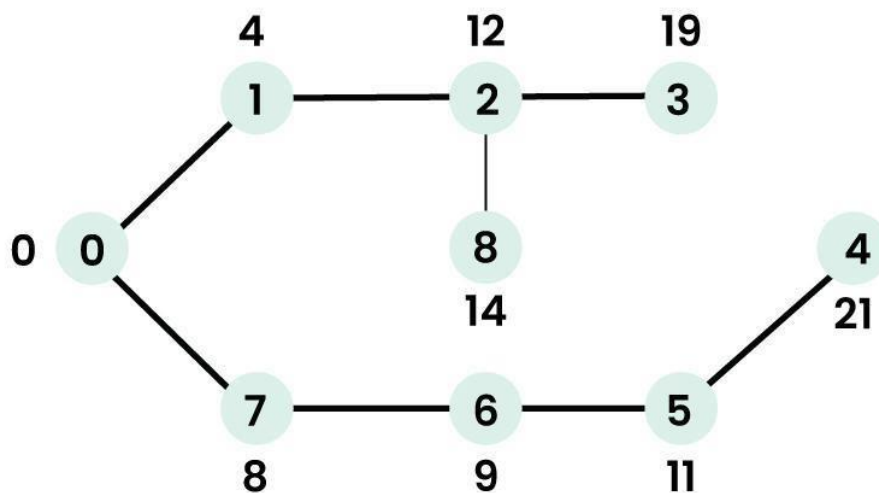
- Pick the vertex with minimum distance value and not already included in **SPT** (not in **sptSET**). Vertex 6 is picked. So **sptSet** now becomes **{0, 1, 7, 6}**.
- Update the distance values of adjacent vertices of 6. The distance value of vertex 5 and 8 are updated.



Working of Dijkstra's Algorithm



We repeat the above steps until *sptSet* includes all vertices of the given graph. Finally, we get the following **Shortest Path Tree (SPT)**.



Working of Dijkstra's Algorithm

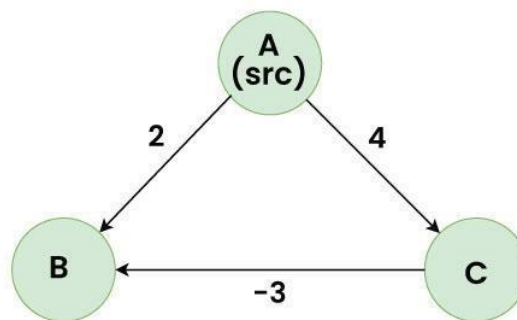


Vertex	Distance from Source
0	0
1	4
2	12
3	19
4	21
5	11
6	9
7	8
8	14

Why Dijkstra's Algorithms fails for the Graphs having Negative Edges ?

The problem with negative weights arises from the fact that Dijkstra's algorithm assumes that once a node is added to the set of visited nodes, its distance is finalized and will not change. However, in the presence of negative weights, this assumption can lead to incorrect results.

Consider the following graph for the example:



Shortest Distance A to B using Dijkstra = 2

Actual Shortest Distance A to B = $4 - 3 = 1$

Failure of Dijkstra in case of negative edges.



In the above graph, A is the source node, among the edges A to B and A to C, A to B is the smaller weight and Dijkstra assigns the shortest distance of B as 2, but because of existence of a negative edge from C to B, the actual shortest distance reduces to 1 which Dijkstra fails to detect.

Note: We use Bellman Ford's Shortest path algorithm in case we have negative edges in the graph.

Pseudocode

ALGORITHM: Dijkstra's Algorithm

INPUT: n, m, u, v, w

INITIALIZE:

graph, dist (set all to infinity)

dist[start] = 0, unvisited.add(start)

WHILE unvisited is not empty:

u = node in unvisited with minimum dist

unvisited.remove(u)

visited.add(u)

for neighbor v of u in graph:

if dist[v] > dist[u] + weight(u, v):

dist[v] = dist[u] + weight(u, v)

if v not in unvisited:

unvisited.add(v)

PRINT dist

Time Complexity

Time Complexity: $O(E * \log V)$, Where E is the number of edges and V is the number of vertices. **Auxiliary Space:** $O(V)$.

Minimum Spanning Tree (MST):

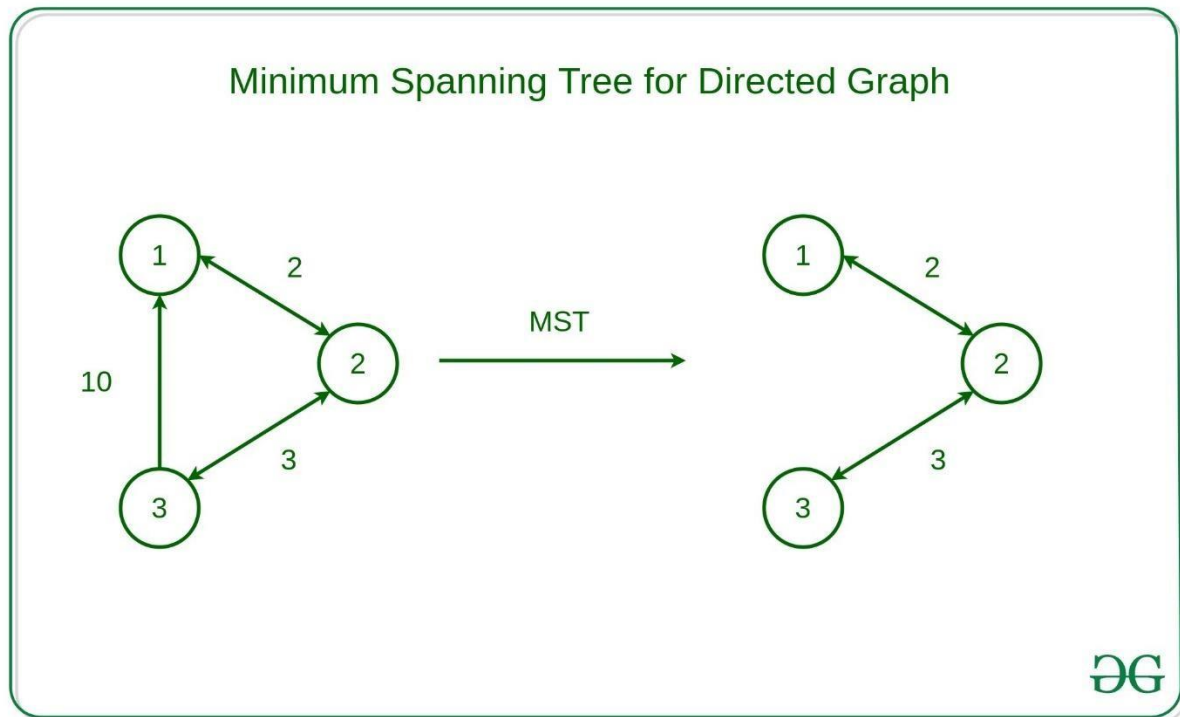
Spanning tree: A **spanning tree** is defined as a tree-like subgraph of a connected, undirected graph that includes all the vertices of the graph.

Properties of a Spanning Tree:

The spanning tree holds the **below-mentioned principles**:

- The number of vertices (**V**) in the graph and the spanning tree is the same.
- There is a fixed number of edges in the spanning tree which is equal to one less than the total number of vertices (**$E = V-1$**).
- The spanning tree should not be **disconnected**, as in there should only be a single source of component, not more than that.
- The spanning tree should be **acyclic**, which means there would not be any cycle in the tree.
- The total cost (or weight) of the spanning tree is defined as the sum of the edge weights of all the edges of the spanning tree.
- There can be many possible spanning trees for a graph.

Minimum spanning tree (MST): A **minimum spanning tree (MST)** is defined as a **spanning tree** that has the minimum weight among all the possible spanning trees.



Algorithms to find Minimum Spanning Tree:

There are several algorithms to find the minimum spanning tree from a given graph, some of them are listed below:

1. Kruskal's minimum spanning tree algorithm.
2. Prime's minimum spanning tree algorithm.

Kruskal's Minimum Spanning Tree (MST)

Algorithm:

How to find MST using Kruskal's algorithm?

Below are the steps for finding MST using Kruskal's algorithm:

1. Sort all the edges in non-decreasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.

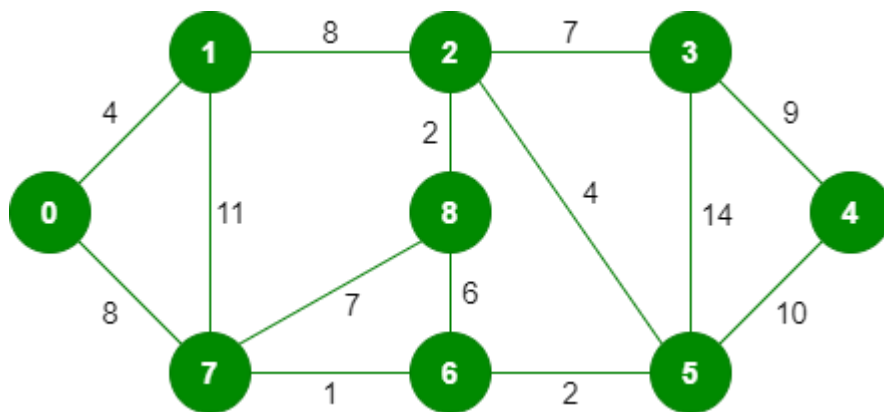
3. Repeat step#2 until there are (V-1) edges in the spanning tree.

Kruskal's algorithm to find the minimum cost spanning tree uses the greedy approach. The Greedy Choice is to pick the smallest weight edge that does not cause a cycle in the MST constructed so far. Let us understand it with an example:

Illustration:

Below is the illustration of the above approach:

Input Graph:



The graph contains 9 vertices and 14 edges. So, the minimum spanning tree formed will be having $(9 - 1) = 8$ edges. After sorting:

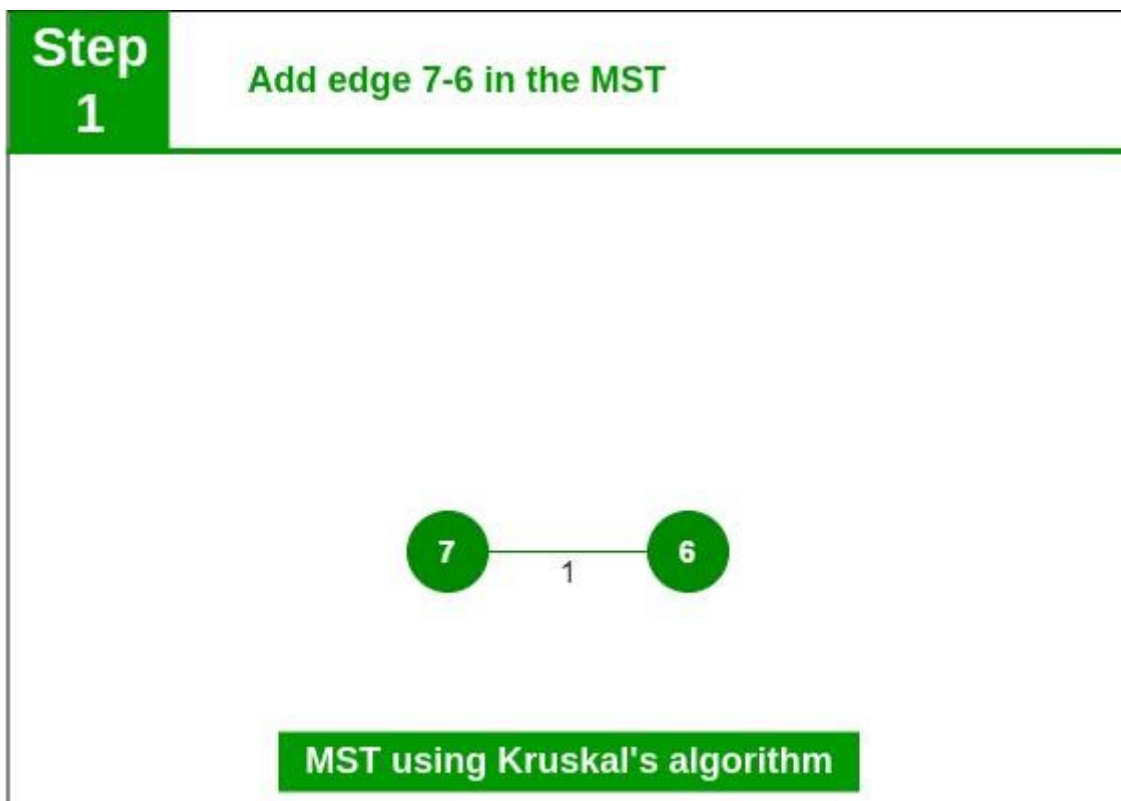
Weight	Source	Destination
1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3

Weight Source Destination

7	7	8
8	0	7
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

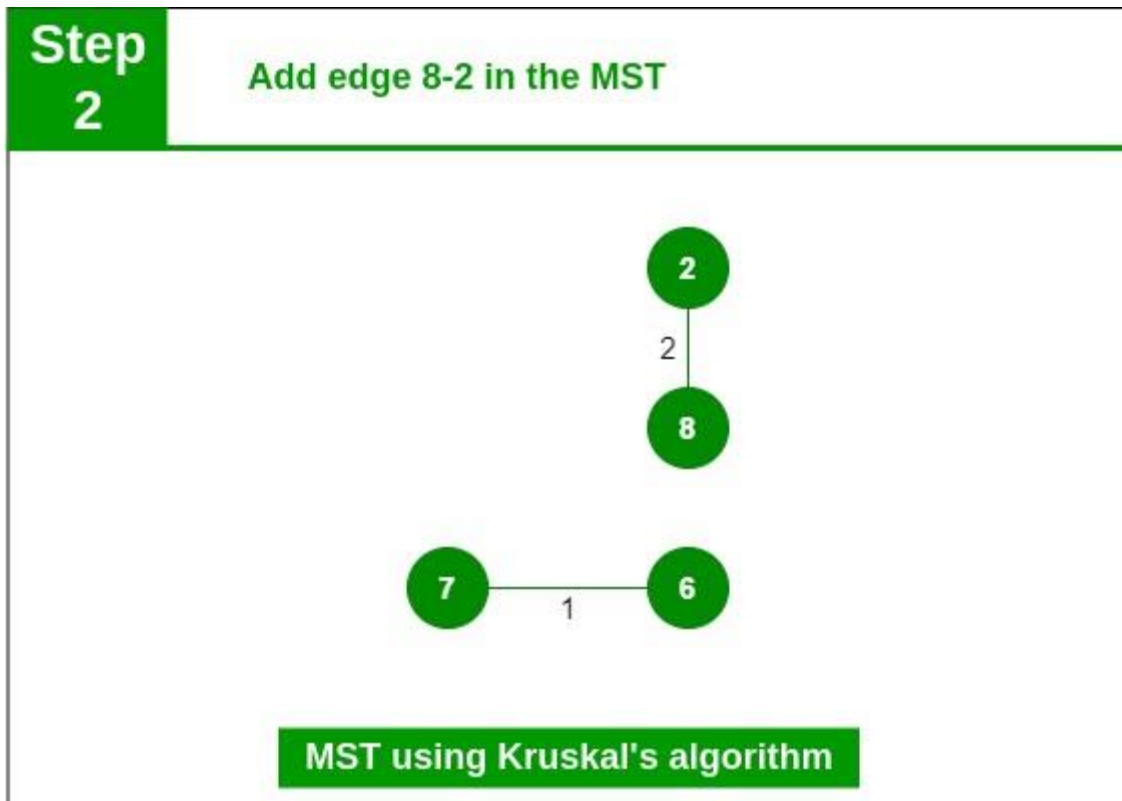
Now pick all edges one by one from the sorted list of edges **Step**

1: Pick edge 7-6. No cycle is formed, include it.



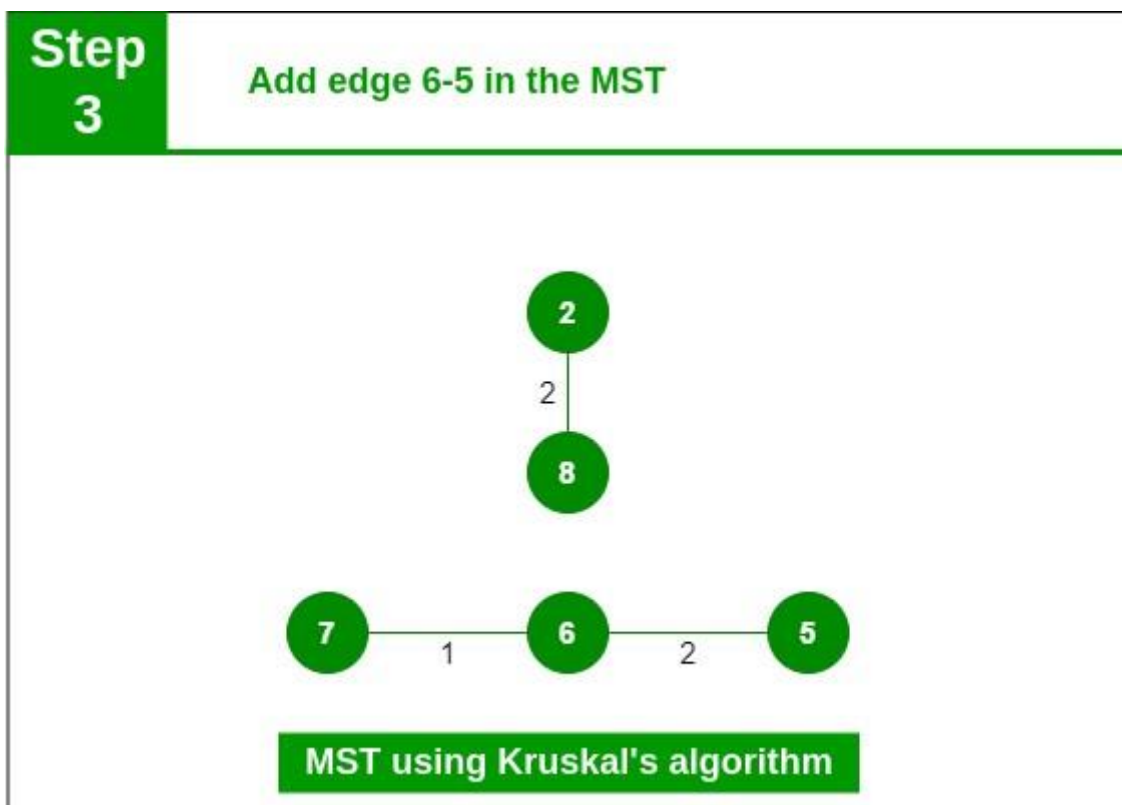
Add edge 7-6 in the MST

Step 2: Pick edge 8-2. No cycle is formed, include it.



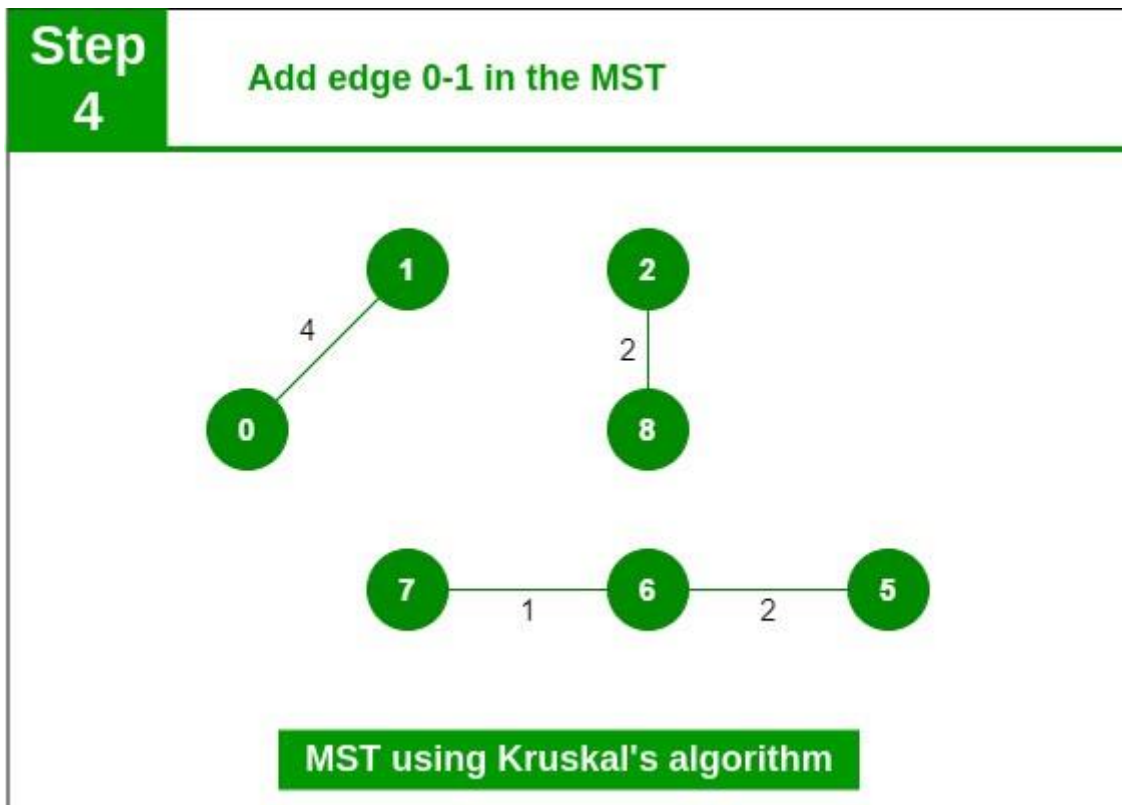
Add edge 8-2 in the MST

Step 3: Pick edge 6-5. No cycle is formed, include it.



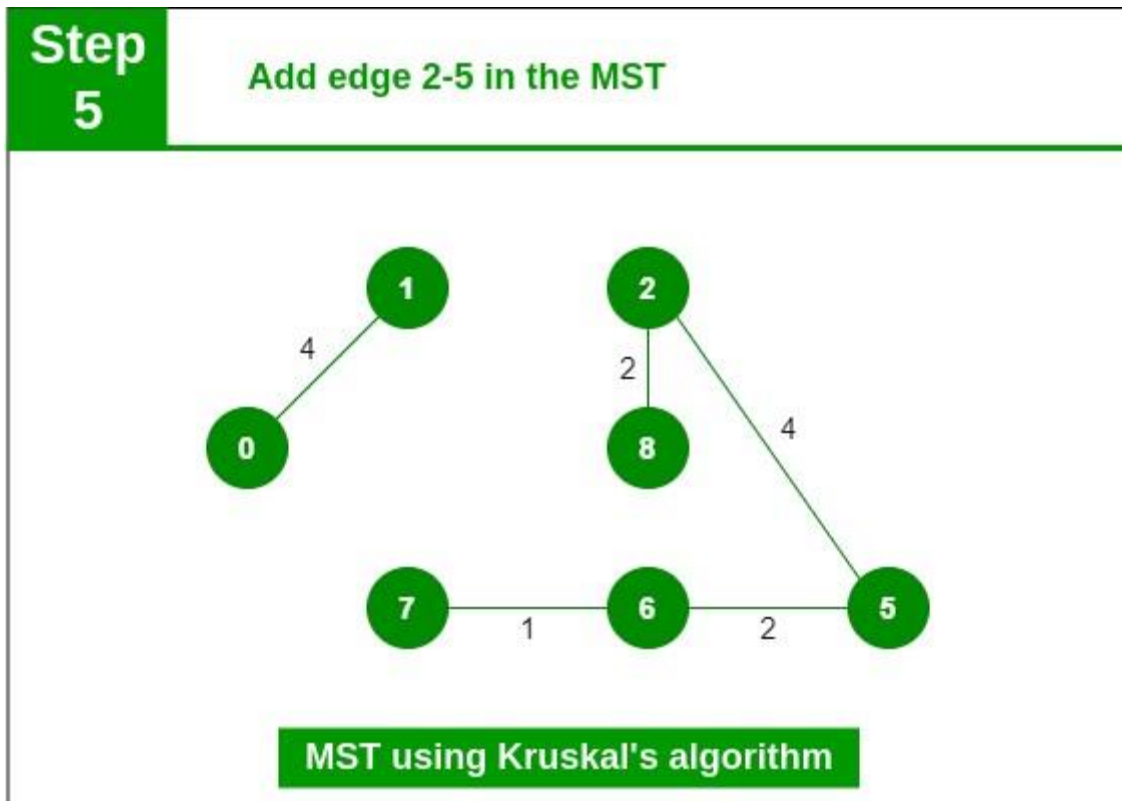
Add edge 6-5 in the MST

Step 4: *Pick edge 0-1.* No cycle is formed, include it.



Add edge 0-1 in the MST

Step 5: *Pick edge 2-5.* No cycle is formed, include it.

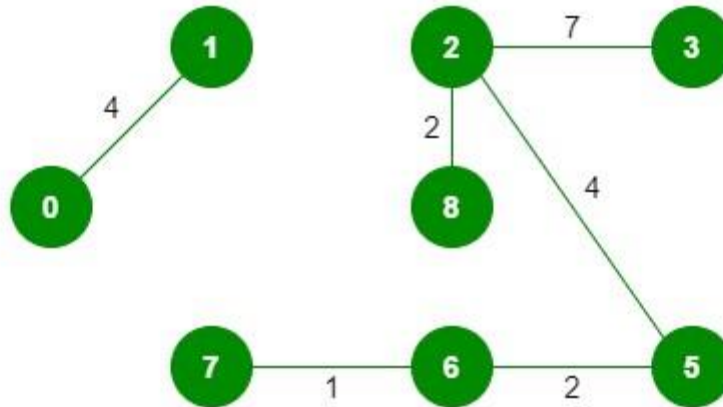


Add edge 2-5 in the MST

Step 6: Pick edge 8-6. Since including this edge results in the cycle, discard it.
Pick edge 2-3: No cycle is formed, include it.

Step 6

Add edge 2-3 in the MST as 8-6 can't be added



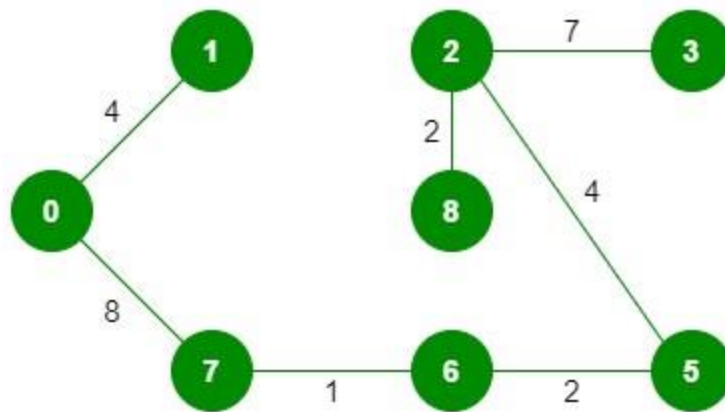
MST using Kruskal's algorithm

Add edge 2-3 in the MST

Step 7: Pick edge 7-8. Since including this edge results in the cycle, discard it.
Pick edge 0-7. No cycle is formed, include it.

Step 7

Add edge 0-7 in the MST as 7-8 can't be added



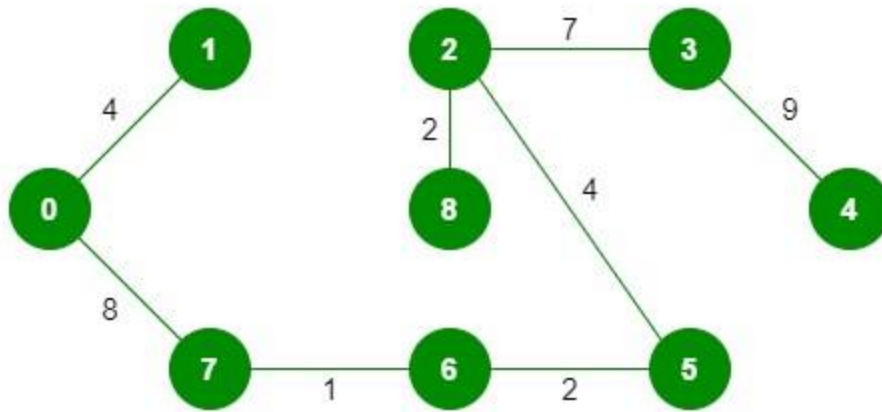
MST using Kruskal's algorithm

Add edge 0-7 in MST

Step 8: Pick edge 1-2. Since including this edge results in the cycle, discard it.
Pick edge 3-4. No cycle is formed, include it.

Step 8

Add edge 3-4 in the MST. It completes the MST



MST using Kruskal's algorithm

Add edge 3-4 in the MST

Note: Since the number of edges included in the MST equals to $(V - 1)$, so the algorithm stops here

Pseudocode

ALGORITHM: Kruskal's Algorithm

INPUT: $n, m, (u, v, w)$

INITIALIZE:

edges = $[(u, v, w)]$ # list of all edges as pairs

parent = $[i \text{ for } i \text{ in range}(n)]$ # parent array for union-find

rank = $[0 \text{ for } i \text{ in range}(n)]$ # rank array for union-find

mst = $[]$ # edges in the minimum spanning tree

SORT edges by weight in ascending order

FOR each (u, v, w) in the sorted edges:

if FIND(parent, u) $\neq$ FIND(parent, v):

UNION(parent, rank, u, v)

mst.append((u, v, w))

if len(mst) == $n-1$:

break # MST found

PRINT mst

Time complexity:

Time Complexity: $O(E * \log E)$ or $O(E * \log V)$

- Sorting of edges takes $O(E * \log E)$ time.
- After sorting, we iterate through all edges and apply the find-union algorithm. The find and union operations can take at most $O(\log V)$ time.
- So overall complexity is $O(E * \log E + E * \log V)$ time.
- The value of E can be at most $O(V^2)$, so $O(\log V)$ and $O(\log E)$ are the same. Therefore, the overall time complexity is $O(E * \log E)$ or $O(E * \log V)$

Auxiliary Space: $O(V + E)$, where V is the number of vertices and E is the number of edges in the graph.

Prim's Algorithm for Minimum Spanning Tree (MST):

How does Prim's Algorithm Work?

The working of Prim's algorithm can be described by using the following steps:

Step 1: Determine an arbitrary vertex as the starting vertex of the MST.

Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

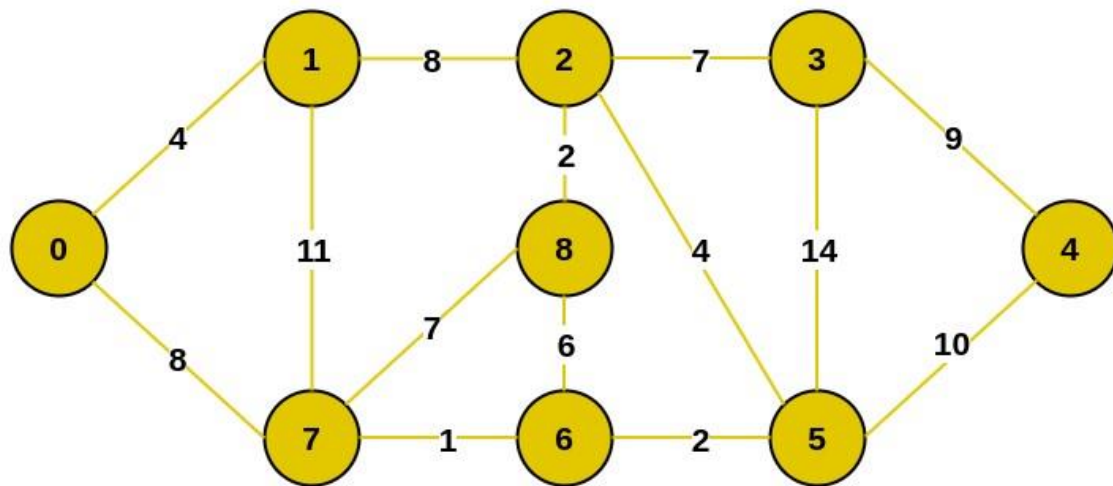
Step 5: Add the chosen edge to the MST if it does not form any cycle.

Step 6: Return the MST and exit

Note: For determining a cycle, we can divide the vertices into two sets [one set contains the vertices included in MST and the other contains the fringe vertices.]

Illustration of Prim's Algorithm:

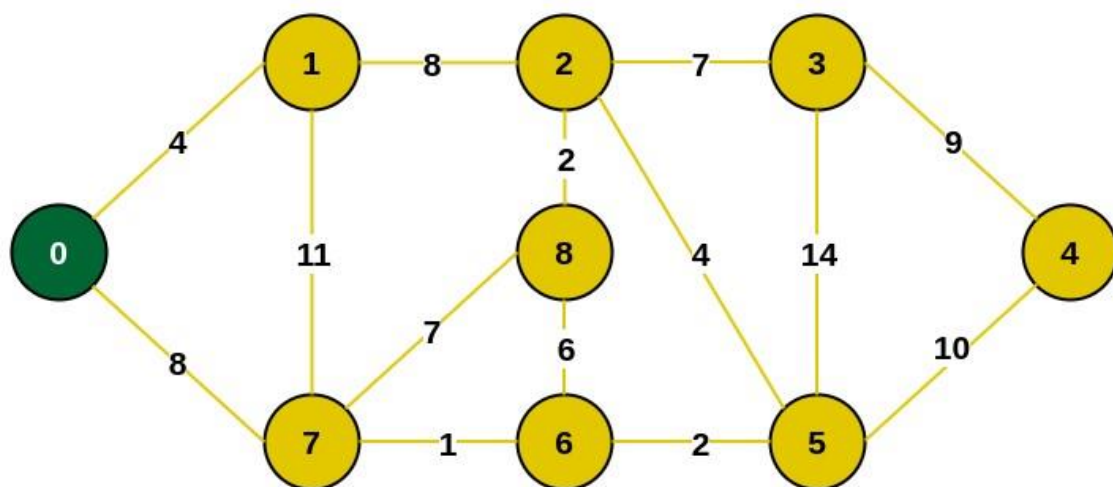
Consider the following graph as an example for which we need to find the Minimum Spanning Tree (MST).



Example of a Graph

Example of a graph

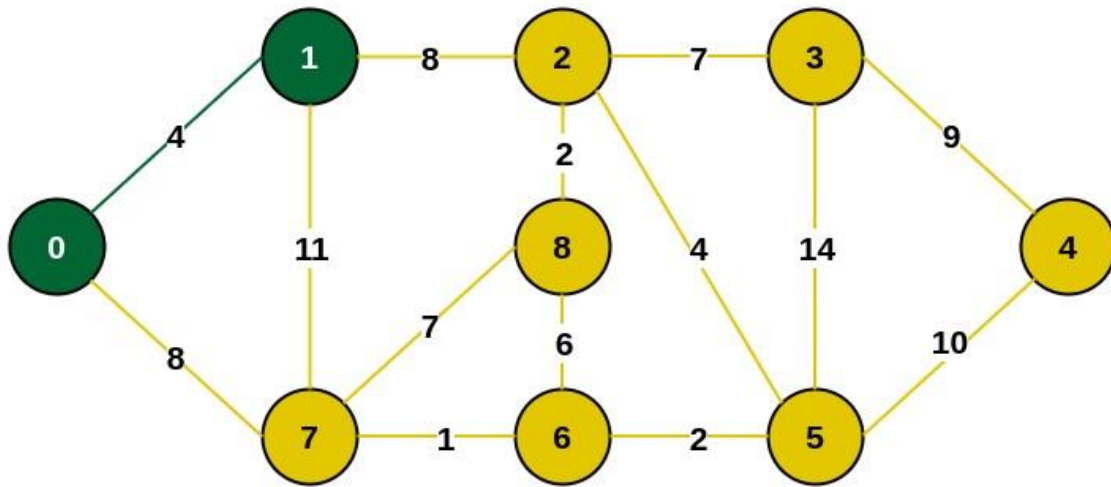
Step 1: Firstly, we select an arbitrary vertex that acts as the starting vertex of the Minimum Spanning Tree. Here we have selected vertex **0** as the starting vertex.



Select an arbitrary starting vertex. Here we have selected 0

0 is selected as starting vertex

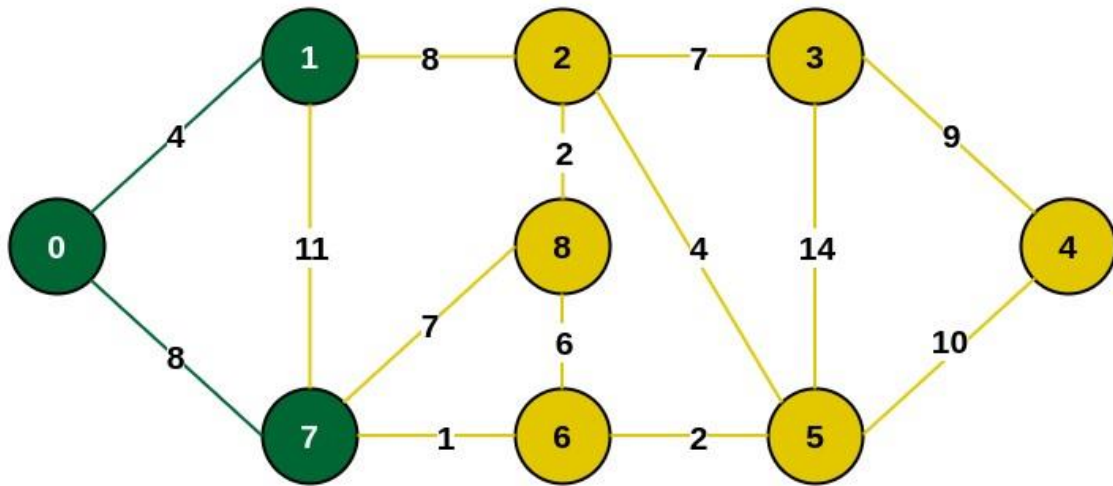
Step 2: All the edges connecting the incomplete MST and other vertices are the edges $\{0, 1\}$ and $\{0, 7\}$. Between these two the edge with minimum weight is $\{0, 1\}$. So include the edge and vertex 1 in the MST.



Minimum weighted edge from MST to other vertices is 0-1 with weight 4

1 is added to the MST

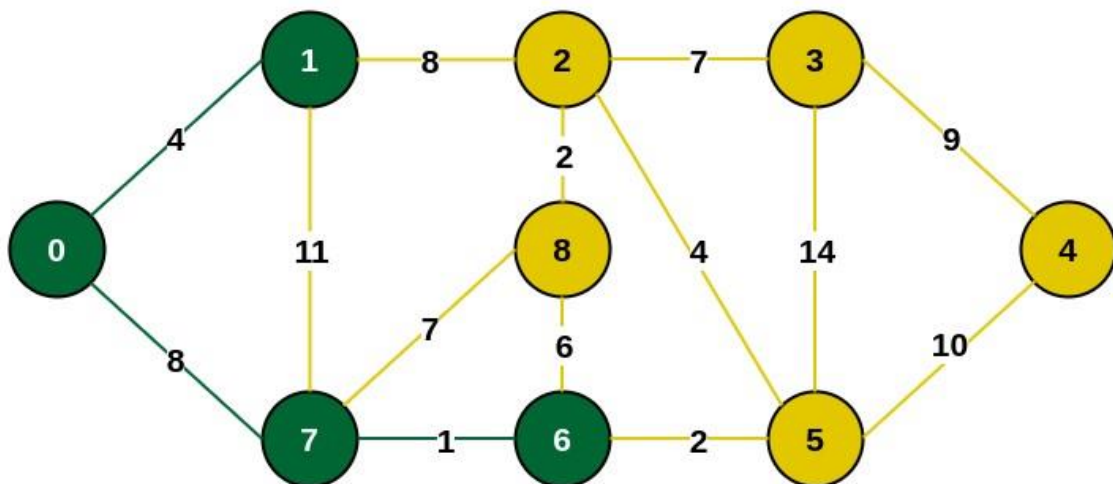
Step 3: The edges connecting the incomplete MST to other vertices are $\{0, 7\}$, $\{1, 7\}$ and $\{1, 2\}$. Among these edges the minimum weight is 8 which is of the edges $\{0, 7\}$ and $\{1, 2\}$. Let us here include the edge $\{0, 7\}$ and the vertex 7 in the MST. [We could have also included edge $\{1, 2\}$ and vertex 2 in the MST].



Minimum weighted edge from MST to other vertices is 0-7 with weight 8

7 is added in the MST

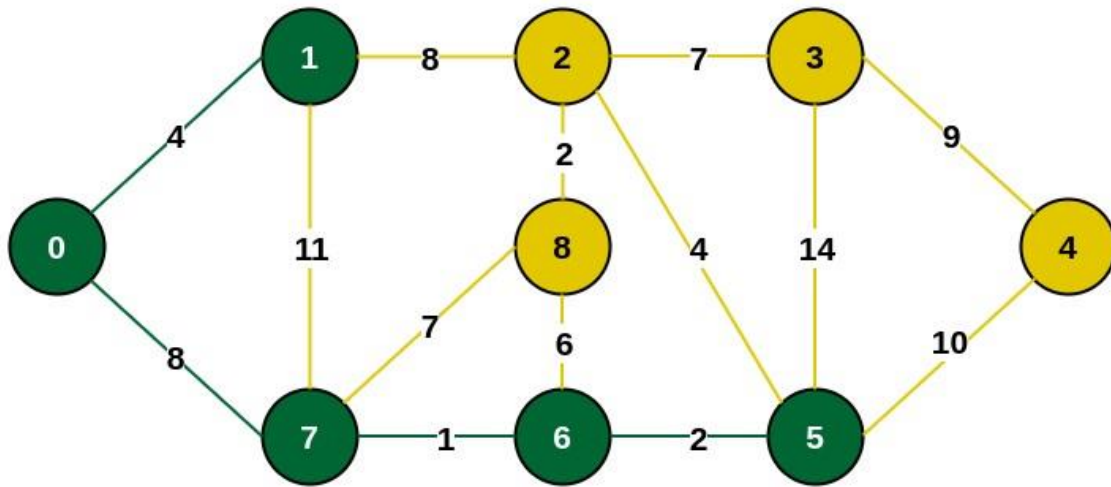
Step 4: The edges that connect the incomplete MST with the fringe vertices are $\{1, 2\}$, $\{7, 6\}$ and $\{7, 8\}$. Add the edge $\{7, 6\}$ and the vertex 6 in the MST as it has the least weight (i.e., 1).



Minimum weighted edge from MST to other vertices is 7-6 with weight 1

6 is added in the MST

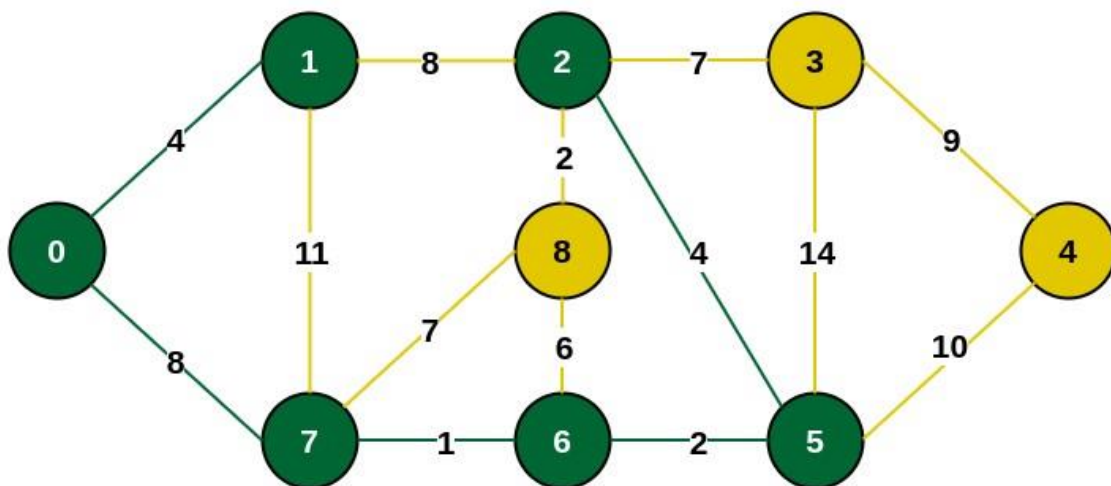
Step 5: The connecting edges now are $\{7, 8\}$, $\{1, 2\}$, $\{6, 8\}$ and $\{6, 5\}$. Include edge $\{6, 5\}$ and vertex 5 in the MST as the edge has the minimum weight (i.e., 2) among them.



Minimum weighted edge from MST to other vertices is 6-5 with weight 2

Include vertex 5 in the MST

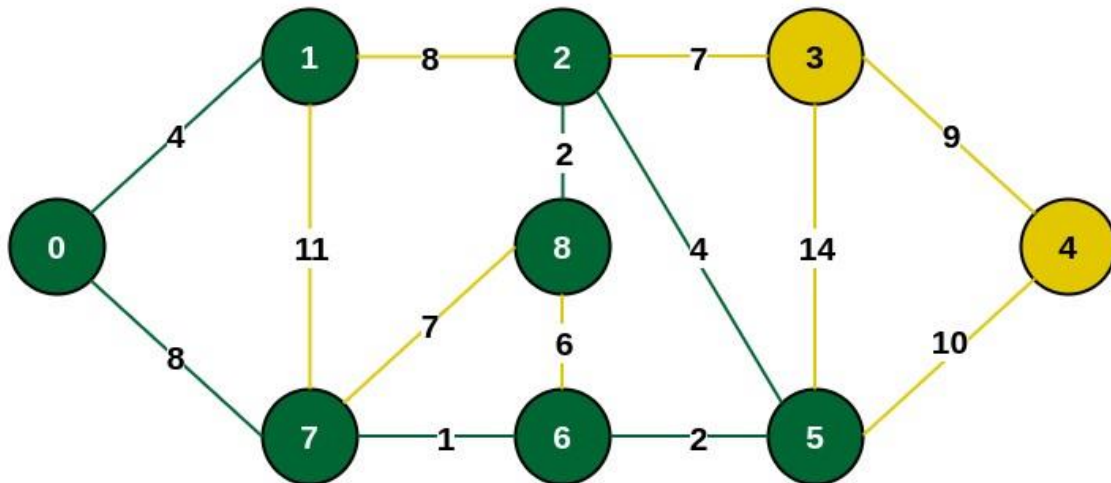
Step 6: Among the current connecting edges, the edge $\{5, 2\}$ has the minimum weight. So include that edge and the vertex 2 in the MST.



Minimum weighted edge from MST to other vertices is 5-2 with weight 4

Include vertex 2 in the MST

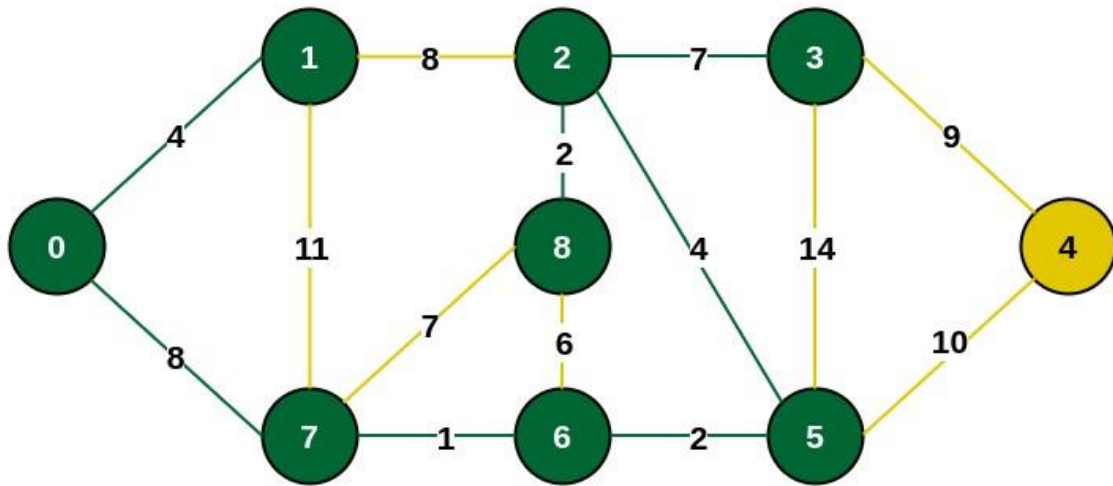
Step 7: The connecting edges between the incomplete MST and the other edges are $\{2, 8\}$, $\{2, 3\}$, $\{5, 3\}$ and $\{5, 4\}$. The edge with minimum weight is edge $\{2, 8\}$ which has weight 2. So include this edge and the vertex 8 in the MST.



Minimum weighted edge from MST to other vertices is 2-8 with weight 2

Add vertex 8 in the MST

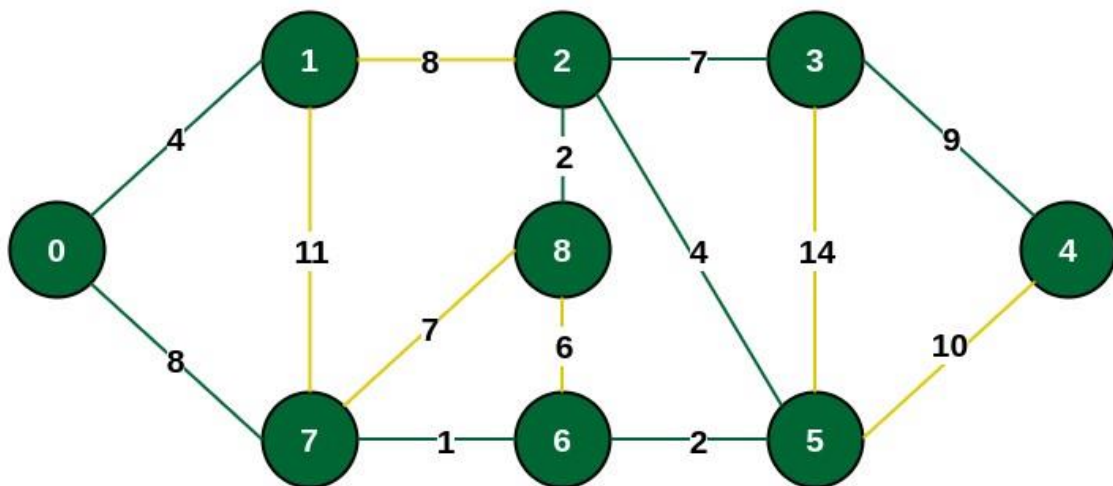
Step 8: See here that the edges $\{7, 8\}$ and $\{2, 3\}$ both have same weight which are minimum. But 7 is already part of MST. So we will consider the edge $\{2, 3\}$ and include that edge and vertex 3 in the MST.



Minimum weighted edge from MST to other vertices is 2-3 with weight 7

Include vertex 3 in MST

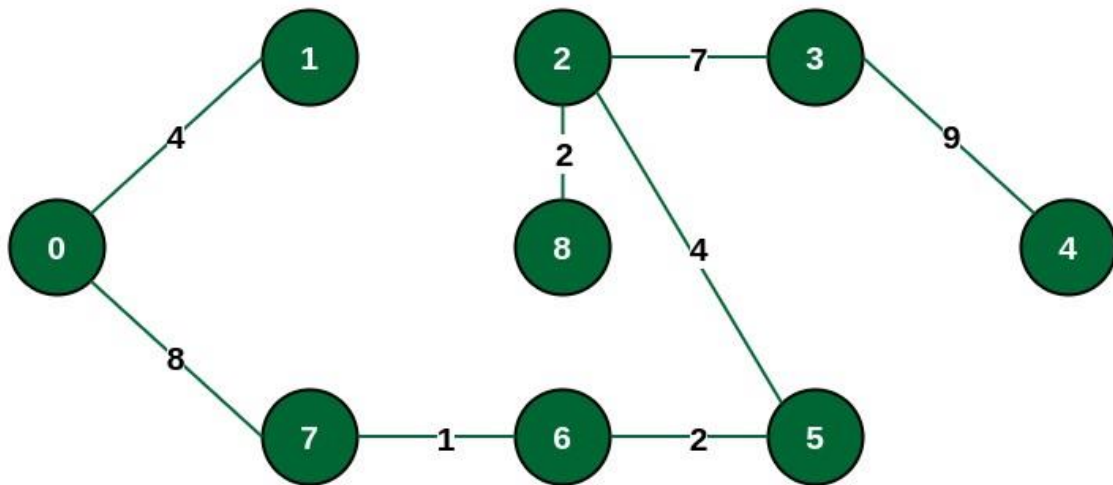
Step 9: Only the vertex 4 remains to be included. The minimum weighted edge from the incomplete MST to 4 is {3, 4}.



Minimum weighted edge from MST to other vertices is 3-4 with weight 9

Include vertex 4 in the MST

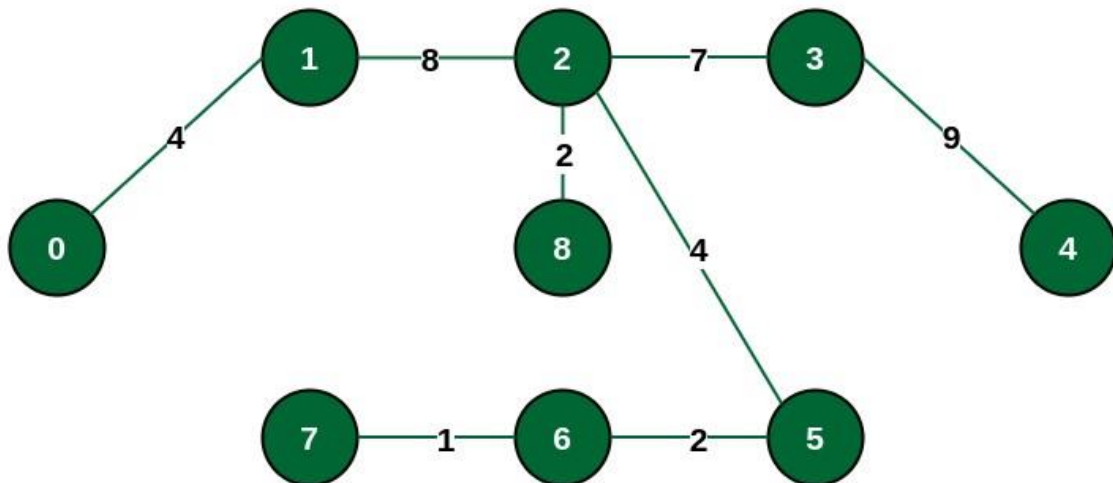
The final structure of the MST is as follows and the weight of the edges of the MST is $(4 + 8 + 1 + 2 + 4 + 2 + 7 + 9) = 37$.



The final structure of MST

The structure of the MST formed using the above method

Note: If we had selected the edge $\{1, 2\}$ in the third step then the MST would look like the following.



Alternative MST structure

Structure of the alternate MST if we had selected edge $\{1, 2\}$ in the MST

Pseudocode:

ALGORITHM: Prim's Algorithm

INPUT: n, m, u, v, w

INITIALIZE:

graph = [(u, v, w)] # list of edges

visited = [False] * n # visited array

dist = [float('inf')] * n # distance array

parent = [-1] * n # parent array

pq = PriorityQueue() # priority queue

dist[start] = 0

pq.push(start, 0)

WHILE pq is not empty:

u = pq.pop()

if visited[u]:

continue

visited[u] = True

for (v, w) in graph[u]:

if not visited[v] and $w < \text{dist}[v]$:

dist[v] = w

`parent[v] = u`

`pq.push(v, dist[v])`

PRINT the minimum spanning tree using the parent array

Time complexity:

Time complexity: $O((n+m) \log n)$, where n is the number of nodes and m is the number of edges.

This is when implemented with a priority queue (e.g., min-heap) to efficiently find the next node to add to the minimum spanning tree.

Huffman Coding

Huffman coding is a greedy approach based lossless data compression algorithm. The main idea in Huffman coding is to assign each character with a variable length code. The length of each character is decided on the basis of its occurrence frequency. The character that occurred most time would have the smallest code and the character that occurred least in the data would have the largest code.

Prefix rule

Huffman coding is implemented using the prefix rule. The variable-length codes assigned to the characters based on their frequency are the **Prefix code**. Prefix codes are used to remove ambiguity during the decoding of the data, no two characters can have the same prefix code.

Major steps in building a Huffman tree

Huffman coding technique involves two major steps, which are as follows:

- Creating a Huffman tree of the input characters.
- Traversing the tree and assigning codes to the characters.

Working or Creation of Huffman Tree

Huffman Coding step-by-step working or creation of Huffman Tree is as follows:

- Step-1: Calculate the frequency of each string.
- Step-2: Sort all the characters on the basis of their frequency in ascending order.
- Step-3: Mark each unique character as a leaf node.
- Step-4: Create a new internal node.
- Step-5: The frequency of the new node as the sum of the single leaf node
- Step-6: Mark the first node as this left child and another node as the right child of the recently created node.
- Step-7: Repeat all the steps from step-2 to step-6.

Formulas Used in Huffman Tree

Average code length per character = $\sum(\text{frequency}_i \times \text{code length}_i) / \sum \text{frequency}_i$

Total number of bits in Huffman encoded message

= Total number of characters in the message x Average code length per character

$$= \sum (\text{frequency}_i \times \text{Code length}_i)$$

Example:

Suppose a data file has the following characters and the frequencies. If Huffman coding is used, calculate:

- Huffman Code of each character
- Average code length
- Length of Huffman encoded data

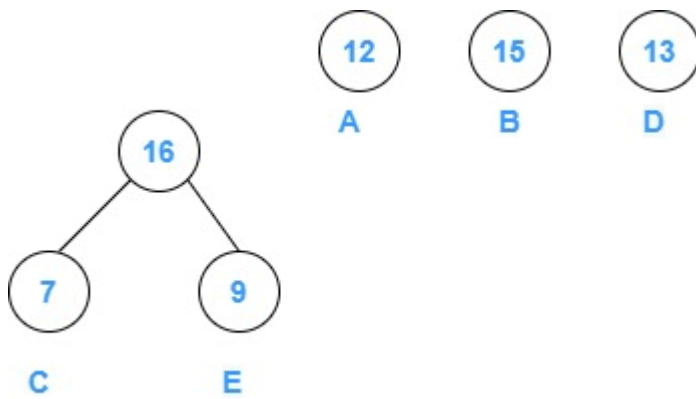
Characters	Frequencies
A	12
B	15
C	7
D	13
E	9

Solution:

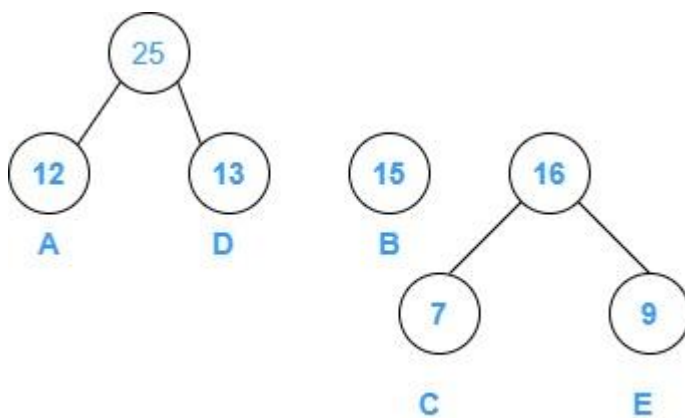
Initially, create the Huffman Tree:



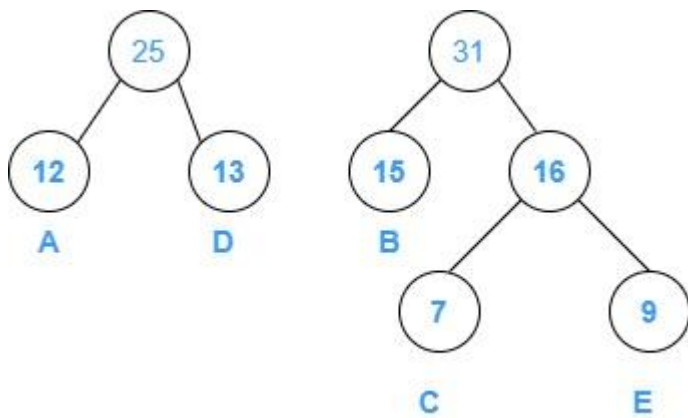
Step-2:



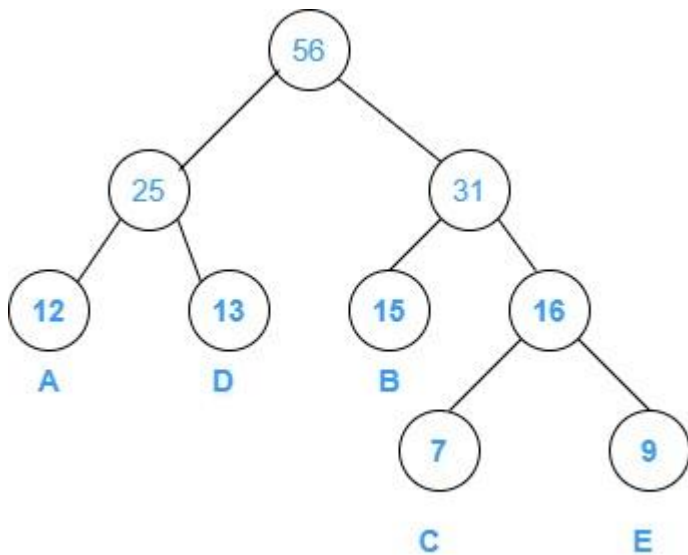
Step-3:



Step-4:



Step-5:

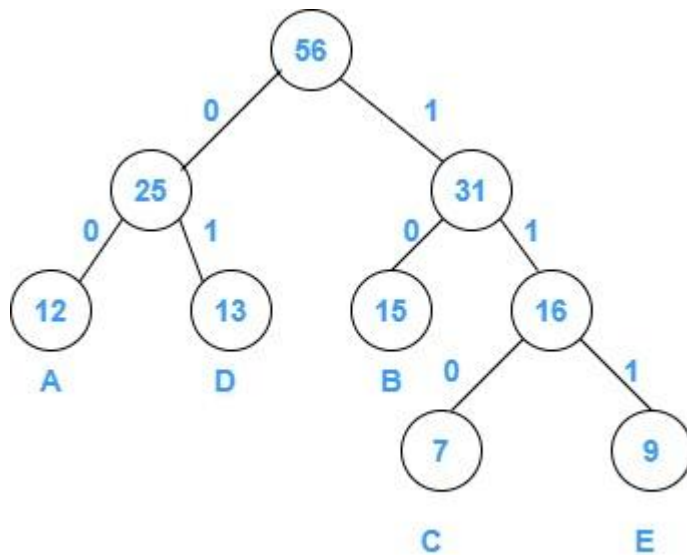


The above tree is a Huffman Tree.

Now, assign weight to all the nodes.

Assign “0” to all left edges and “1” to all right edges.

The tree will become



Huffman Code of each character:

A: 00

B: 10

C: 110

D: 01

E: 111

$$\begin{aligned}
 \text{Average code length per character} &= \frac{\sum (\text{frequency}_i \times \text{code length}_i)}{\sum \text{frequency}_i} \\
 &= \{(12 \times 2) + (13 \times 2) + (15 \times 2) + (7 \times 3) + (9 \times 3)\} / (12 + 13 + 15 + 7 + 9) \\
 &= (24 + 26 + 30 + 21 + 27) / 56 \\
 &= 128 / 56 \\
 &= 2.28
 \end{aligned}$$

Average code length per character = 2.28

Total number of bits in Huffman encoded message

= Total number of characters in the message x Average code length per character

$$= 56 \times 2.28$$

$$= 127.68$$

Algorithm for Huffman coding

For implementing Huffman coding, we need a priority queue that will have all the unique characters of the data. Then sort all the characters on the basis of their occurring frequency in ascending order. Take out the minimum value from the priority queue and assign it to the left child and similarly assign the value to the right child.

Pseudocode for the Huffman coding

Input: A set of unique characters and their frequencies

Output: The Huffman coding tree

Steps:

1. Create a priority queue X to store the unique characters
2. Sort the characters in X in ascending order based on their frequencies
3. While there is more than one node in X:
 - a. Create a new node (newNode)
 - b. Extract the two nodes with the smallest frequencies from X
 - c. Assign the extracted nodes as the left and right child of newNode
 - d. Assign the sum of the frequencies of the two extracted nodes as the value of newNode
 - e. Insert newNode into X
4. The remaining node in X is the root of the Huffman coding tree
5. Return the root node

Complexity

Time complexity is $O(n \log n)$.

Application of Huffman coding.

- Huffman coding is used in traditional compression techniques like GZIP etc.
- For text and fax transmissions